

"Ss. Cyril and Methodius" University in Skopje
**FACULTY OF COMPUTER
SCIENCE AND ENGINEERING**

Семинарска работа · Seminar Report

Cognitive Robotics / Cognitive Architectures · Doctoral studies

Memory-Augmented Cognitive Architectures for Reinforcement-Learning Agents

A hybrid symbolic–neural design (CARMA), with a controlled study of whether — and how — episodic memory improves a deliberating agent

Agon Bajgora — PhD candidate, Faculty of Computer Science and Engineering (FINKI), Ss. Cyril and Methodius University, Skopje

Cognitive Robotics / Cognitive Architectures · Supervisor: Prof. Dr. Andrea Kulakov · Skopje, June 2026 (revised October 2026) · Code: <https://github.com/Agonb/carma>

[Abstract](#)

[1. Introduction](#)

[2. Background: three traditions, used as inspiration](#)

[3. The CARMA architecture](#)

[4. Methodology — “how to test it”](#)

[5. Experiment 1 — does memory improve the agent?](#)

[6. Experiment 2 — does it matter how memory is organised? \(equal budget\)](#)

[7. Honest assessment of what this is](#)

[8. Bridge to the thesis: one memory architecture, two decision domains](#)

[9. Conclusion and next steps](#)

[Code and data availability](#)

[References](#)

Abstract

This seminar studies a single, concrete question from the cognitive-architecture literature: *does adding memory measurably improve a deliberating agent, and does it matter how that memory is organised?* I propose **CARMA**, a hybrid symbolic–neural cognitive architecture for a reinforcement-learning agent — neural perception (autoencoder), an episodic memory, a reinforcement-learning decision core, and a Soar-inspired symbolic supervisor — explicitly assembled as a *synthesis* of established traditions (Soar, OpenCog Hyperon, memory-augmented reinforcement learning). I then build a small, fully reproducible, CPU-only prototype and run two controlled experiments. The first confirms, with a clean no-memory-vs-memory ablation over 20 seeds, that an episodic replay memory makes the agent reach competence roughly three times faster — a deliberate re-demonstration of a textbook effect, used here as a validated instrument. The second, an *equal-budget* comparison of four memory-selection mechanisms, asks the harder question and returns an honest negative result: a learned-similarity (“analogy”) key built from the autoencoder latent does **not** beat plain uniform replay when the environment is fully observable. A third experiment adds the smallest working version of the architecture’s *symbolic* half — a rule-based safety gate (promote/reject/escalate) that, in a catastrophic-hazard variant, provably cuts hazard entries by ~60% while *improving* task success, so the agent is genuinely hybrid symbolic–neural rather than only on paper. I discuss why (the learned key only re-encodes already-known state), what it implies (the hypothesised benefit belongs to partially observable settings), and where a genuine research contribution would live: transferring a regime-keyed, *auditable* episodic memory into an abstention-capable autonomous decision system (the CGX trading architecture). Throughout, I separate what is *demonstrated* from what is *designed*, and what is *known* from what would be *new*.

1. Introduction

A central theme of cognitive architectures — from Soar and ACT-R to modern neural-symbolic systems — is that intelligent behaviour is organised around **memory**: working memory for the current situation, episodic memory for specific past experiences, and semantic memory for distilled regularities. A natural and testable claim follows: an agent that can store and reuse its experience should behave more intelligently than an otherwise identical agent that cannot.

This seminar takes that claim literally and tests it. Rather than survey the field, I (i) specify a concrete hybrid symbolic–neural architecture that uses the components named in our discussions — autoencoders for perception, an LSTM for temporal anticipation, an episodic memory, and a Soar-style symbolic control layer — and (ii) build a small but complete prototype that lets memory be switched on and off, and reorganised, under controlled conditions. The guiding question, in plain terms:

Does giving a reinforcement-learning agent a memory make it learn faster — and does it matter how the agent chooses which memories to reuse?

I have tried to be scrupulous about what is established versus what is new. The headline “memory helps” result is a known effect from the reinforcement-learning literature; I reproduce it deliberately, as a trustworthy measuring instrument, and say so. The more interesting scientific content is a controlled comparison of memory *mechanisms* and an honest negative result, plus a diagnosed failure mode that clarifies *why* memory helps.

2. Background: three traditions, used as inspiration

Soar (symbolic). Soar organises cognition around a working-memory blackboard, production rules that propose and prefer *operators* (the only units of deliberate action), a decision cycle, and *impasses* that spawn subgoals and drive learning by *chunking*. I borrow its structure and its commitment to interpretable, traceable action selection — not its implementation.

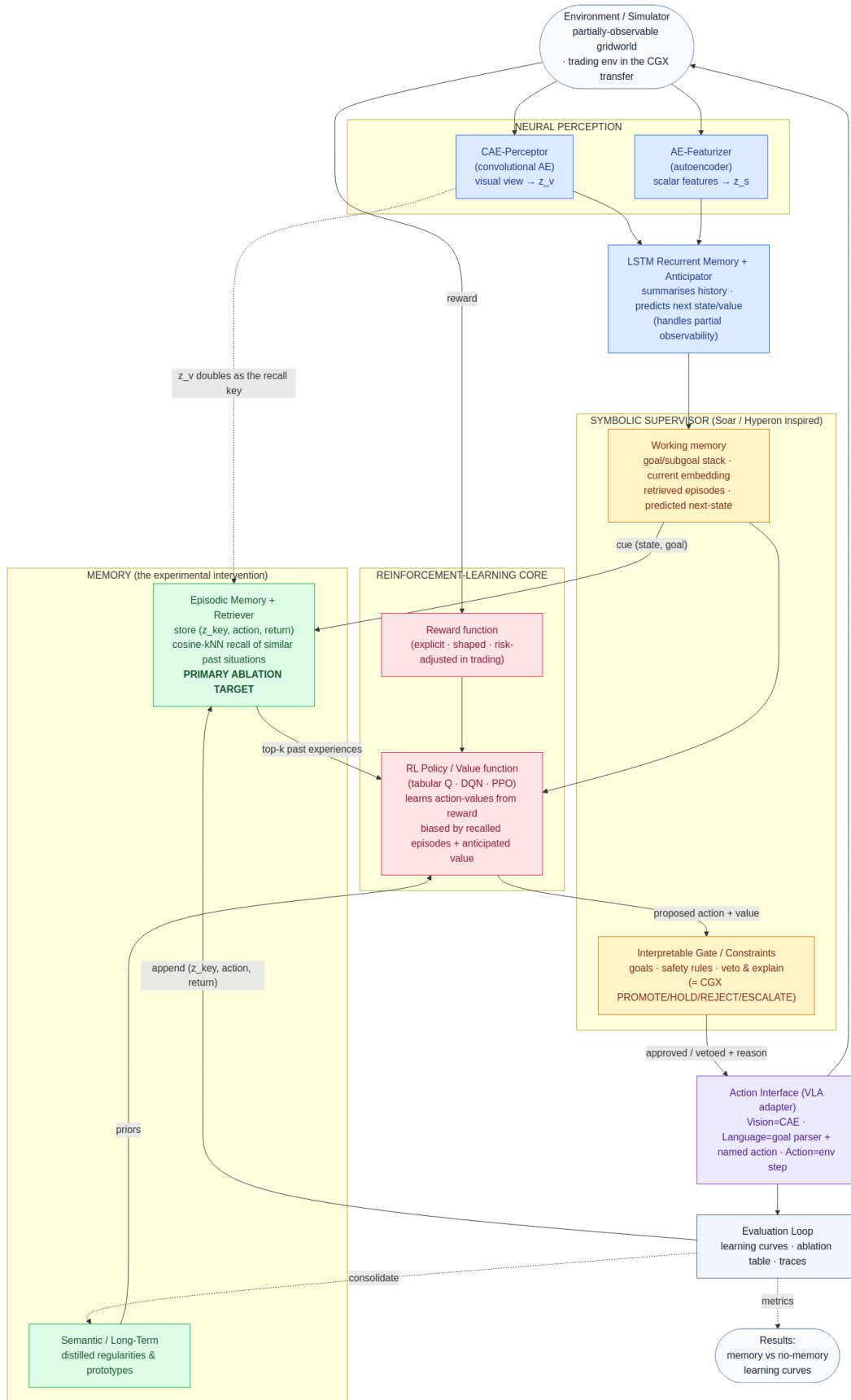
OpenCog Hyperon (neural-symbolic). Hyperon (Goertzel et al., 2023, arXiv:2310.18318) — a near-ground-up *rewrite* of OpenCog — argues for a single typed substrate (the Atomspace metagraph, queried by MeTTa-style pattern matching) in which symbolic facts and sub-symbolic/neural representations coexist. I borrow this stance — one shared representation read by both the learner and the memory — at laptop scale, as inspiration rather than re-implementation.

Memory-augmented reinforcement learning. Storing and replaying past experience to learn faster is one of the most established ideas in RL: experience replay (Lin, 1992; popularised at scale by Mnih et al., 2015), Dyna-style model-based planning (Sutton, 1990), prioritized sweeping / backward focusing (Moore & Atkeson, 1993), prioritized experience replay (Schaul et al., 2016), and episodic control (Blundell et al., 2016); see Sutton & Barto (2018) for the unifying taxonomy. The reverse-replay sweep used below is closest to prioritized sweeping, and is independently motivated by hippocampal reverse replay (Foster & Wilson, 2006; Mattar & Daw, 2018). The recurrent handling of partial observability (Hausknecht & Stone, 2015) motivates the LSTM.

CARMA is a deliberate *combination* of these, not a new mechanism. I state this plainly because a synthesis presented honestly is defensible, whereas a relabelled standard system presented as novel is not.

3. The CARMA architecture

CARMA (Cognitive Architecture for Reinforcement-learning, Memory, and Anticipation) is a deliberating agent whose neural perception and anticipation feed a reinforcement-learning decision core, wrapped by a symbolic supervisor that keeps it interpretable.



The CARMA architecture. Neural perception (autoencoders) and an LSTM anticipator populate a working memory; an episodic memory supplies recalled experience; a reinforcement-learning core

selects actions from reward; a Soar-inspired symbolic supervisor applies goals and constraints and can veto and explain.

Components: **CAE/autoencoder perception** compresses observations into a latent code; **LSTM anticipation** predicts the next state (the “anticipative actor” idea); **episodic memory** stores and recalls past experience; the **reinforcement-learning core** learns to act from reward; and a **symbolic supervisor/gate** applies goals and hard constraints and can veto an action and record a human-readable reason. The invariant that preserves interpretability is that *actions are chosen by the learned policy under symbolic supervision* — every decision can be traced.

3.1 What is demonstrated vs what is designed (stated up front)

A diagram is not running code. The prototype in this seminar implements a well-defined *slice* of CARMA; the rest is design and planned future work:

Component	Status in this prototype
Environment (gridworld, random start)	implemented
Autoencoder perception	implemented ; <i>load-bearing</i> only in the similarity-replay mechanism (Section 6)
Episodic memory (4 retrieval mechanisms)	implemented
Reinforcement-learning core (tabular Q-learning)	implemented
Convolutional autoencoder (CAE)	designed; prototype uses a plain autoencoder
LSTM anticipation	designed; not yet implemented
Symbolic supervisor / gate	implemented as a minimal safety gate (Section 6.2: PROMOTE/REJECT/ESCALATE with logged reasons); the full Soar-style supervisor is still designed
Vision-language-action interface	designed; not yet implemented
Transfer to autonomous decision-making (CGX)	designed; Section 8

This honesty is deliberate: it lets the demonstrated result be judged on its own terms and keeps the architecture’s claims separate from the experiment’s evidence.

4. Methodology — “how to test it”

Formally the task is a Markov decision process (states = maze cells, actions = {up, down, left, right}, discount $\gamma = 0.98$, reward as below); the episodic memory is a store of past transitions (s, a, r, s') with a retrieval function that decides which to replay, and the experiments manipulate exactly that perception/retrieval machinery.

The environment is a 15×15 maze generated by a seeded algorithm (identical every run). The agent starts at a **random** cell each episode, so it must learn to reach the goal from *anywhere* — making

credit assignment across the whole state space, rather than memorising one path, the bottleneck. The agent perceives the maze, the autoencoder compresses that perception, and a tabular Q-learning core learns action values from a reward (reaching the goal is rewarded; steps and traps are penalised).

The perceptual encoder is the “autoencoder” primitive of this course: a one-hidden-layer network (ReLU encoder, linear decoder) with a 12-unit latent, trained with Adam to reconstruct the observation under mean-squared error on transitions gathered by random exploration. Its latent is stored with each transition and serves as the episodic-memory key; a convolutional autoencoder is the planned Phase-2 drop-in (Section 3.1).

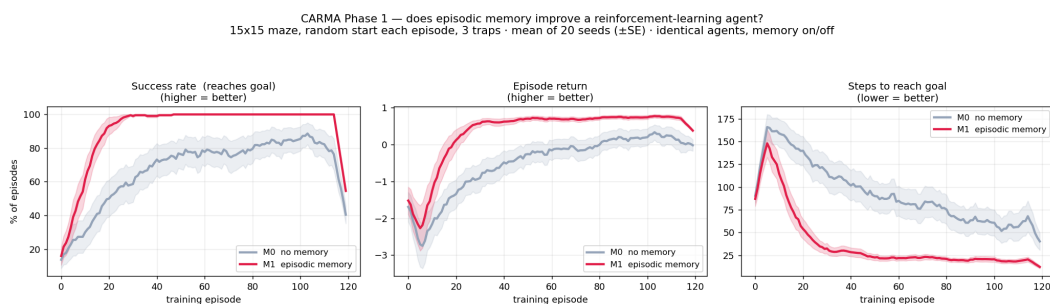
Three controlled comparisons, each holding everything except the manipulated factor (perception, reward, exploration schedule, environment, random seeds) fixed:

1. **Does memory help?** (Section 5) — identical agents with memory on vs off.
2. **Does it matter how memory is organised?** (Section 6) — identical agents and an *identical replay budget*, varying only the rule by which past experiences are selected.
3. **Does a symbolic safety gate help?** (Section 6.2) — the trained policy with a rule-based supervisor (promote/reject/escalate) off vs on, in a catastrophic-hazard maze variant.

Reported metrics: success rate (reaching the goal), episode return, steps to the goal, episodes to 90% success (sample efficiency), and a per-step decision trace (interpretability). All curves are means over many seeds with standard-error bands.

5. Experiment 1 — does memory improve the agent?

Two identical Q-learning agents differ by one switch. **M0** has no memory and learns only from the single transition it is currently on. **M1** keeps an episodic memory and replays stored experience — uniform replay each step plus a *reverse replay* of each finished episode (inspired by hippocampal reverse replay; Foster & Wilson, 2006), which propagates the goal’s value back along the path in one sweep.



Experiment 1: identical agents, memory on vs off, mean of 20 seeds (±SE). The memory-augmented agent reaches high success far sooner and converges to a shorter path.

metric (last 25 episodes)	M0 — no memory	M1 — episodic memory
success rate	82%	100%

metric (last 25 episodes)	M0 — no memory	M1 — episodic memory
episode return	+0.15	+0.73
steps to reach goal	62	20
success after only 20 episodes	50%	90%
episodes to 90% success	47	16

The memory-augmented agent reaches competence roughly **three times faster** (episodes to 90% success: 47.5 vs 15.6; ratio 3.05×). This gap is statistically reliable, not seed noise: paired across the 20 shared seeds the difference is 31.9 episodes (bootstrap 95% CI [26.4, 37.4]), paired $t(19) = 11.2$, $p \approx 8\text{e-}10$. I want to be explicit about the mechanism and the honesty of the comparison: M1 performs more value-updates per real step *because* it has stored experience to replay; M0 cannot, because it has no memory. That is not a confound to apologise for — it is precisely the point of memory (more learning from the same real experience), and the fair axis is performance per episode of *real* interaction, which is what the curves show. Equally, this is a deliberate re-demonstration of a known effect (experience/reverse replay improves sample efficiency), used as a validated instrument — not a new finding.

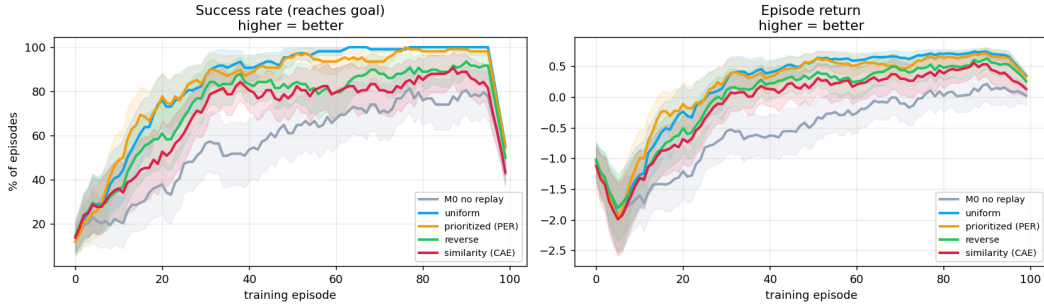
5.1 A diagnosed failure (why this design, not another)

An earlier design used the *recalled returns* of similar past situations to bias action selection directly. It made memory **hurt**: estimating an action’s value as the best return ever observed nearby produces values that violate Bellman consistency, and a greedy policy over them oscillates between states (loops), so the agent performed worse than with no memory at all. Diagnosing this and replacing it with value-consistent replay is, I think, the most instructive part of the work: it shows the benefit of memory depends on *how* it enters the decision, not merely on its presence.

6. Experiment 2 — does it matter how memory is organised? (equal budget)

“Memory helps” is known. The sharper question is whether the *selection mechanism* matters once we remove the “more compute” advantage. I fix the replay budget — the same number of value-updates per real step for every replay arm — and vary only how the replayed experiences are chosen: **uniform** (random; standard experience replay), **prioritized** (insertion-time |TD-error|; an approximation of PER, without priority refresh or importance-sampling weights), **reverse** (most-recent, in reverse), and **similarity (CAE)** — the k nearest past situations in the autoencoder latent space (analogy by similarity). The last is the mechanism built from the cognitive ingredients of this course, and the only one in which the autoencoder is load-bearing.

CARMA — equal-budget memory-mechanism comparison
15x15 random-start maze · 15 replay updates/step for every replay arm · mean of 12 seeds ($\pm$ SE)



Experiment 2: four memory mechanisms at equal replay budget, mean of 12 seeds ($\pm$ SE). All replay arms beat no-replay; uniform and prioritized are fastest; the learned-similarity key is the weakest replay arm here.

mechanism (equal replay budget)	final success	episodes to 90% success
no replay (floor)	77%	74
uniform	100%	29
prioritized (PER)	98%	29
reverse	91%	41
similarity (CAE)	86%	50

An honest negative result. The learned-similarity key does *not* beat plain uniform replay — it is in fact the weakest replay arm, and the gap is statistically reliable across the 12 shared seeds, not noise (similarity is +20.8 episodes-to-90% slower than uniform; paired $t(11) = 3.8$, $p = 0.003$, bootstrap 95% CI [10.7, 31.3]). The reason is instructive and not a bug: this maze is fully observable, so the autoencoder latent mostly re-encodes the agent’s already-known position and adds nothing over uniform selection; focusing replay on perceptually-near experiences is even slightly myopic. The clear implication is that the hypothesised advantage of a learned similarity key belongs to **partially observable** settings with perceptual aliasing — where the agent cannot see its true state and a learned representation must do real work. That is a specific, falsifiable next experiment (egocentric view + the LSTM), not a hand-wave. What this experiment *did* establish: the autoencoder is now load-bearing and *tested* rather than assumed, and we have learned — by measurement — precisely when a structured memory does and does not earn its place.

6.1 Interpretability

Because action selection is traceable, every decision of the trained agent can be explained. A sample greedy trace (`decision_trace.txt`) shows the agent moving along a monotonically increasing value gradient toward the goal, with the chosen action and its learned value at each step. I deliberately do not overclaim this: a clean value gradient is generic to any converged tabular Q-learner and does not yet *cite the memory*. The intended interpretability layer — the symbolic

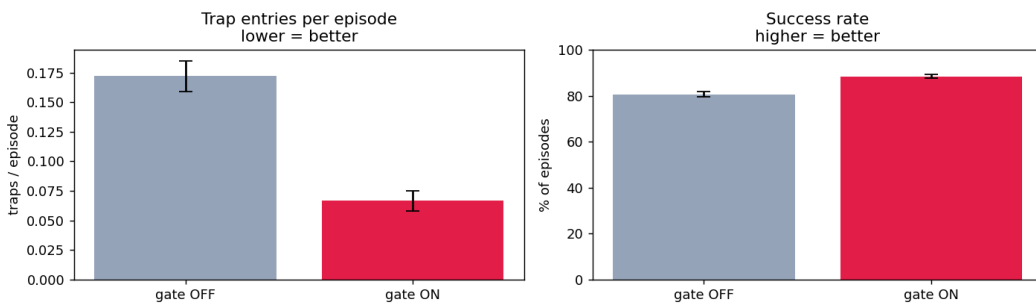
supervisor surfacing the *recalled episodes* that justify or veto each action — is designed, not yet demonstrated (Section 3.1). A first attempt to surface the nearest recalled episodes at decision time was itself instructive: in this fully-observable maze the autoencoder-latent recall is dominated by the constant scene and returns near-identical neighbours regardless of position — the same degeneracy that makes similarity replay underperform above, and further evidence that a cited-episode audit trail becomes meaningful only when observations vary richly (partial observability, and the market-regime keys of the CGX transfer in Section 8).

6.2 A minimal symbolic safety gate (PROMOTE / REJECT / ESCALATE)

The experiments above exercise the *learning* half of CARMA. To make the architecture genuinely hybrid I add the smallest working version of its *symbolic* half: a rule-based supervisor between the learned policy and the environment that enforces one explicit safety constraint — *never enter a known hazard tile* — emitting a human-readable decision in the CGX vocabulary at every step: **PROMOTE** (the policy’s action is safe — execute it), **REJECT** (it would enter a hazard — veto it and substitute the safe action that best progresses toward the goal), or **ESCALATE** (no safe progress exists — flag it). The supervisor’s knowledge (the hazard set and the layout) is symbolic and separate from the learned Q-values, so it enforces something the learner is never guaranteed to respect.

A safety gate is only meaningful where hazards are *unacceptable*, so this study makes two deliberate changes from Experiments 1–2 (only here): hazards are **catastrophic** (entering one ends the episode) and the maze is **braided** (loops give redundant routes, so safe detours can exist). I evaluate 12 trained agents over 60 episodes each at $\epsilon = 0.25$ (the residual exploration/uncertainty a deployed policy faces), gate off vs on.

CARMA — minimal symbolic safety gate over the learned policy
12 trained agents, 60 eval episodes at $\epsilon=0.25$ (mean $\pm$ SE)



Experiment 3: a minimal symbolic safety gate over the learned policy. Catastrophic-hazard braided maze, 12 agents, 60 eval episodes at $\epsilon = 0.25$ (mean $\pm$ SE). The gate cuts hazard entries and, because hazards are catastrophic, raises task success.

metric ($\epsilon = 0.25$)	gate OFF	gate ON
hazard entries / episode	0.172	0.067
success rate	81%	88%

metric ($\epsilon = 0.25$)	gate OFF	gate ON
episode return	+0.34	+0.46

The gate **provably changes outcomes**: hazard entries fall by **61%** (paired across the 12 seeds, $t(11) = 6.9$, $p \approx 3e-5$), and because the hazards are catastrophic, preventing them is pure benefit — success rises $81 \rightarrow 88\%$ and return $+0.34 \rightarrow +0.46$. Only 0.8% of decisions were vetoes and 48 were escalations (a hazard genuinely unavoidable from that cell); every one is logged with its reason (`gate_trace.txt`), e.g. “*vetoed UP $\rightarrow$ hazard at (8,11); substituted safe goal-progressing action LEFT.*” This is the same promote/reject/escalate-with-reason machinery CGX needs at its trading gate (Section 8), realised here in its smallest honest form; the full Soar-style supervisor — goal/subgoal stack, learned constraints, escalation-on-impasse — remains designed (Section 3.1).

7. Honest assessment of what this is

As a coursework deliverable I believe this is solid: every primitive from our discussion is present in a coherent design; there is a complete, reproducible, from-scratch prototype (the agent, environment, autoencoder and memory are pure numpy); the experiments are controlled, seeded, and tested for statistical significance; and I report a negative result and a diagnosed failure rather than only the favourable curve.

As a *research contribution*, I am explicit that it is not one yet. Experiment 1 reproduces a textbook effect. CARMA is a synthesis of known components, not a new mechanism. The one piece that could have been distinctive — the learned-similarity memory key — was tested fairly and did not beat the standard baseline in this setting. I would rather state this plainly than overclaim, because the value of the work is the controlled understanding it produces, and a clear map of what remains open.

7.1 Limitations

In one place: the results are on a single fixed maze layout (varied only over random start cells and seeds), so external validity to other layouts is untested; the learner is tabular Q over the true state, i.e. fully observable, which is precisely why the learned-similarity key has no room to help here; a full Soar-style symbolic supervisor (beyond the minimal safety gate of Section 6.2), the LSTM, the convolutional autoencoder and the vision-language-action interface are designed but not yet built (Section 3.1); the prioritized arm is an approximation of PER; and significance rests on 12–20 seeds. None of these affect the two reported results — both reproduce exactly and are statistically tested — but they scope what the work does and does not yet establish.

8. Bridge to the thesis: one memory architecture, two decision domains

This seminar is also the foundation of the doctoral topic I would like to propose: **“Regime-Keyed Episodic Memory: A Cognitive Memory Architecture for Memory-Augmented Autonomous Agents, Evaluated in an Embodied Cognitive-Robotic Simulation and a Non-Embodied**

Decision Domain.” The claim is that the mechanism studied here — a memory keyed by the agent’s own learned representation of its *situation* (regime), whose recalled precedents feed an interpretable act/hold/abstain gate — is a **domain-general cognitive primitive**, not a task-specific trick. The embodied gridworld of this seminar is the primary, controlled domain (Phase 2 adds the partial-observability + LSTM test where the learned key should earn its place). The **non-embodied validation domain already exists**: CGX, our multi-agent trading system (Bajgora, Kulakov & Merovci, under revision at *Electronics*), whose market-regime classifier and deterministic promote/hold/reject/escalate gate are the same architecture without a body — currently **stateless**. The thesis contribution is to give that gate an **episodic memory of past regimes**, so each decision recalls analogous past situations and *cites them as the audit trail* for acting or abstaining. The contribution is not sample efficiency or “beating the market” — it is **memory in the service of governance and interpretability**, evaluated on abstention quality and calibration over time. The maze in this seminar is the de-risked sandbox for the mechanism; the two domains together are what make it a thesis.

9. Conclusion and next steps

I specified a hybrid symbolic–neural memory-augmented architecture, built a reproducible prototype, and used it to (1) confirm, as a controlled instrument, that episodic memory improves a reinforcement-learning agent’s sample efficiency ($\sim 3\times$); (2) show, at equal budget, that *how* memory is organised matters — and that a learned-similarity key does not help when the state is fully observable; and (3) add a minimal symbolic safety gate that provably cuts catastrophic-hazard entries ($\sim 60\%$) while improving success, making the agent genuinely hybrid. Concrete next steps: (a) the partial-observability experiment (egocentric perception + LSTM) where a learned key should earn its place; (b) growing the safety gate into the full Soar-style supervisor (goal stack, learned constraints, escalation-on-impasse); and (c) the thesis target — the same regime-keyed, auditable episodic memory evaluated in both domains: the embodied simulation (primary) and the CGX decision system (validation), per the topic proposal accompanying this report.

Code and data availability

All code, configurations and the results reported here are public at <https://github.com/Agonb/carma> (MIT licence). The repository contains the gridworld, the autoencoder, the episodic memory with its four replay mechanisms, the Q-learning agent and the symbolic safety gate, plus one script per experiment (`run_experiment.py`, `run_mechanisms.py`, `run_gate.py`) that regenerates the figures, summaries and statistics in `results/`. Everything runs on a laptop CPU with NumPy, SciPy and Matplotlib.

References

- Goertzel, B., et al. (2023). *OpenCog Hyperon: A Framework for AGI at the Human Level and Beyond*. arXiv:2310.18318.

- Laird, J. E. (2012). *The Soar Cognitive Architecture*. MIT Press.
- Lin, L.-J. (1992). *Self-improving reactive agents based on reinforcement learning, planning and teaching*. Machine Learning, 8.
- Sutton, R. S. (1990). *Integrated architectures for learning, planning, and reacting (Dyna)*. ICML.
- Moore, A. W., & Atkeson, C. G. (1993). *Prioritized Sweeping: Reinforcement Learning with Less Data and Less Time*. Machine Learning, 13.
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
- Mnih, V., et al. (2015). *Human-level control through deep reinforcement learning (DQN)*. Nature, 518.
- Schaul, T., et al. (2016). *Prioritized Experience Replay*. ICLR.
- Blundell, C., et al. (2016). *Model-Free Episodic Control*. arXiv:1606.04460.
- Hausknecht, M., & Stone, P. (2015). *Deep Recurrent Q-Learning for Partially Observable MDPs*. AAAI Fall Symposium.
- Foster, D. J., & Wilson, M. A. (2006). *Reverse replay of behavioural sequences in hippocampal place cells during the awake state*. Nature, 440.
- Mattar, M. G., & Daw, N. D. (2018). *Prioritized memory access explains planning and hippocampal replay*. Nature Neuroscience, 21.